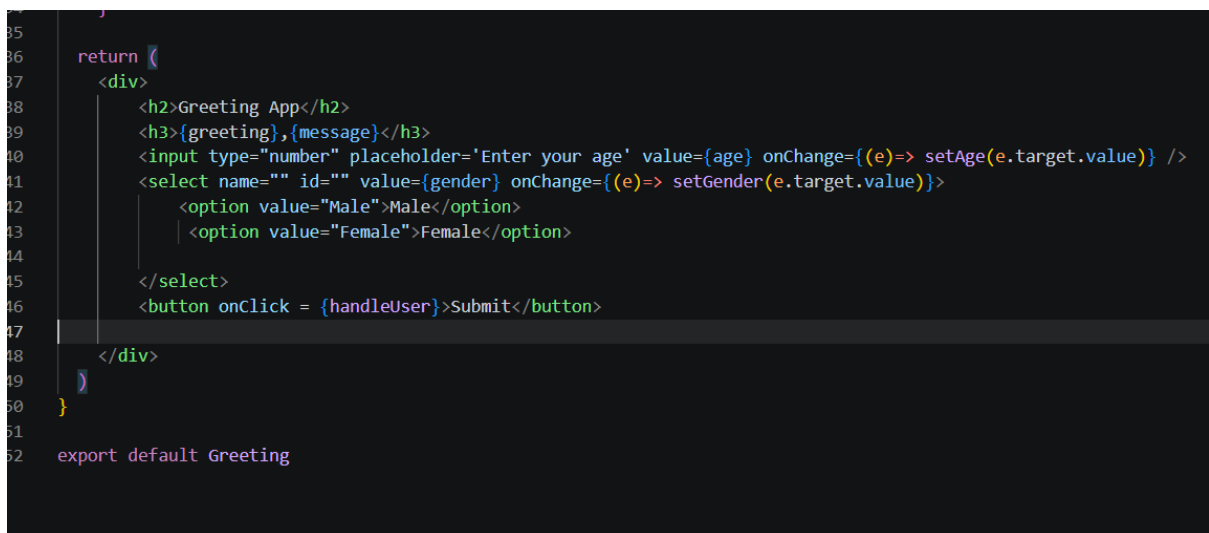
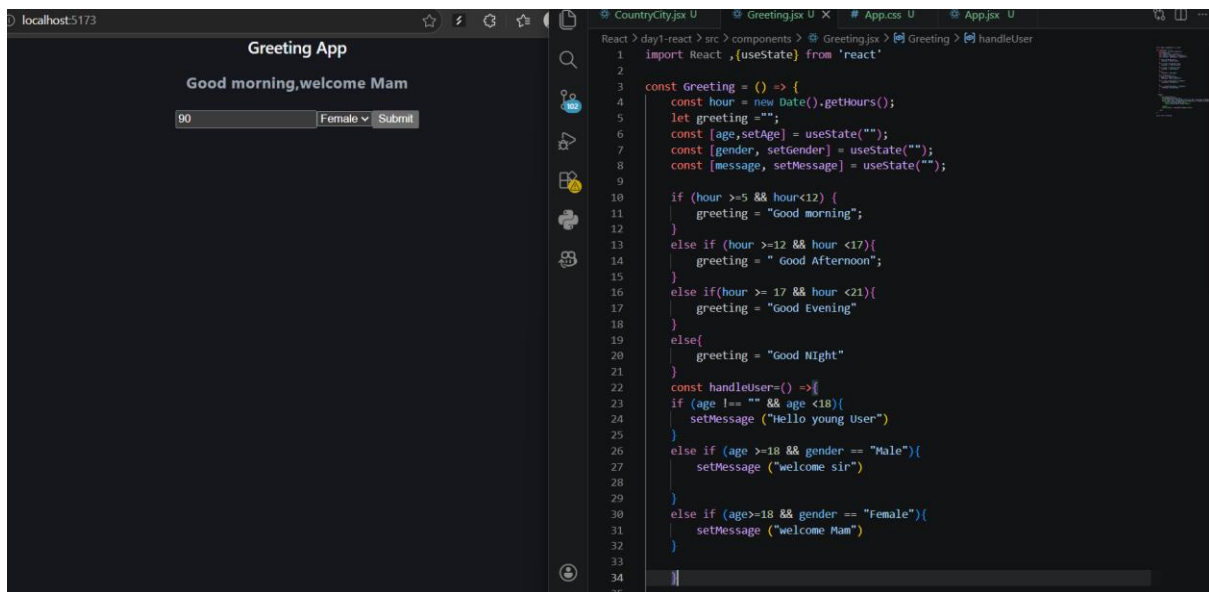
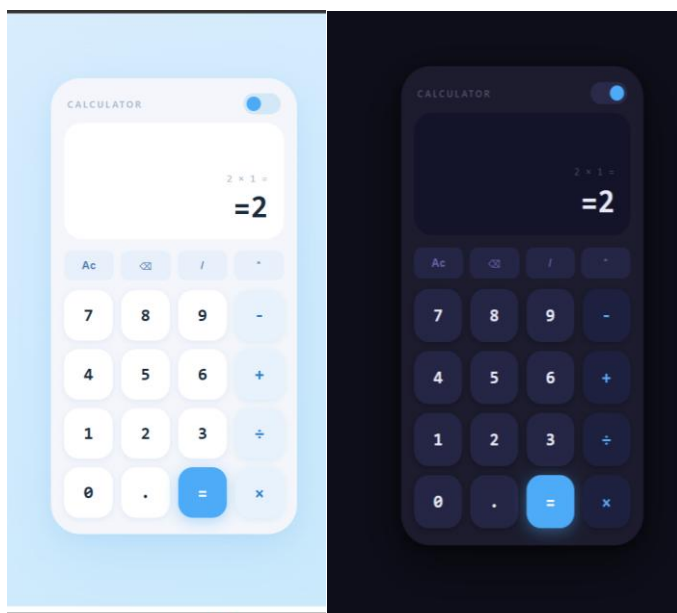




Calculator



```
React > day1-react > src > components > Calculator.jsx > Calculator > handleDecimal
1 import React, { useState } from "react";
2
3 const calculator = () => {
4   const [display, setDisplay] = useState("0");
5   const [expression, setExpression] = useState("");
6   const [firstValue, setFirstValue] = useState(null);
7   const [operator, setOperator] = useState(null);
8   const [waitingForSecond, setWaitingForSecond] = useState(false);
9   const [isDark, setIsDark] = useState(false);
10
11   const handleNumberClick = (num) => {
12     if (display === "0" || waitingForSecond) {
13       setDisplay(num);
14       setWaitingForSecond(false);
15     } else if (display.length < 12) {
16       setDisplay(display + num);
17     }
18   };
19
20   const handleDecimal = () => {
21     if (waitingForSecond) {
22       setDisplay("0.");
23       setWaitingForSecond(false);
24       return;
25     }
26     if (!display.includes(".")) setDisplay(display + ".");
27   };
28
29   const handleOperatorClick = (op) => {
30     if (operator && !waitingForSecond) {
31       handleEqualClick(true);
32       setTimeout(() => {
33         setFirstValue(Number(display));
34         setOperator(op);
35         setWaitingForSecond(true);
36         setExpression(display + " " + op);
37       }, 0);
38     }
39   };
40
41   const handleEqualClick = (chain = false) => {
42     if (firstValue === null || operator === null) return;
43     const secondValue = Number(display);
44     let result = 0;
45     if (operator === "+") result = firstValue + secondValue;
46     else if (operator === "-") result = firstValue - secondValue;
47     else if (operator === "x") result = firstValue * secondValue;
48     else if (operator === "÷") {
49       if (secondValue === 0) {
50         setDisplay("Error");
51         setExpression("");
52         setFirstValue(null);
53         setOperator(null);
54         setWaitingForSecond(false);
55         return;
56       }
57       result = firstValue / secondValue;
58     }
59     if (!chain) setExpression(firstValue + " " + operator + " " + secondValue + "=");
60     const rounded = parseFloat(result.toFixed(10));
61     setDisplay(String(rounded));
62     setFirstValue(null);
63     setOperator(null);
64     setWaitingForSecond(false);
65   };
66
67   const handleClear = () => {
68     setDisplay("0");
69     setExpression("");
70     setFirstValue(null);
71     setOperator(null);
72     setWaitingForSecond(false);
73   };
74
75   return (
76     <div>
77       <input type="checkbox" checked="" /> Dark Mode
78     </div>
79     <div>
80       <input type="text" value={display} />
81       <div>
82         <button>C</button>
83         <button>CE</button>
84         <button>AC</button>
85       </div>
86       <div>
87         <button>+</button>
88         <button>-</button>
89         <button>x</button>
90         <button>÷</button>
91       </div>
92       <div>
93         <button>1</button>
94         <button>2</button>
95         <button>3</button>
96         <button>4</button>
97         <button>5</button>
98         <button>6</button>
99         <button>7</button>
100        <button>8</button>
101        <button>9</button>
102        <button>0</button>
103        <button>.</button>
104        <button>+</button>
105        <button>-</button>
106        <button>x</button>
107        <button>÷</button>
108        <button>=</button>
109      </div>
110    </div>
111  );
112}
```

```
    return;
  }
  setExpression(display + " " + op);
  setFirstValue(Number(display));
  setOperator(op);
  setWaitingForSecond(true);
};

const handleEqualClick = (chain = false) => {
  if (firstValue === null || operator === null) return;
  const secondValue = Number(display);
  let result = 0;
  if (operator === "+") result = firstValue + secondValue;
  else if (operator === "-") result = firstValue - secondValue;
  else if (operator === "x") result = firstValue * secondValue;
  else if (operator === "÷") {
    if (secondValue === 0) {
      setDisplay("Error");
      setExpression("");
      setFirstValue(null);
      setOperator(null);
      setWaitingForSecond(false);
      return;
    }
    result = firstValue / secondValue;
  }
  if (!chain) setExpression(firstValue + " " + operator + " " + secondValue + "=");
  const rounded = parseFloat(result.toFixed(10));
  setDisplay(String(rounded));
  setFirstValue(null);
  setOperator(null);
  setWaitingForSecond(false);
};

const handleClear = () => {
  setDisplay("0");
  setExpression("");
  setFirstValue(null);
  setOperator(null);
  setWaitingForSecond(false);
};
```

```

const handleClear = () => {
  setDisplay("0");
  setExpression("");
  setFirstValue(null);
  setOperator(null);
  setWaitingForSecond(false);
};

const handleBackspace = () => {
  if (waitingForSecond || display === "Error") { setDisplay("0"); return; }
  setDisplay(display.length > 1 ? display.slice(0, -1) : "0");
};

const handleButtonClick = (btn) => {
  if (!isNaN(btn)) handleNumberClick(btn);
  else if (btn === ".") handleDecimal();
  else if (btn === "C") handleClear();
  else if (btn === "⌫") handleBackspace();
  else if (btn === "=") handleEqualClick(false);
  else handleOperatorClick(btn);
};

const t = isDark ? theme.dark : theme.light;

const topButtons = ["c", "μ", "sin", "deg"];
const mainButtons = [
  "7", "8", "9", "-",
  "4", "5", "6", "+",
  "1", "2", "3", "÷",
  "0", ".", "=", "⌫",
];

const isOp = (b) => ["+", "-", "⌫", "÷"].includes(b);
const isEq = (b) => b === "=";
const isFunc = (b) => ["c", "μ", "sin", "deg", "Ac", "⌫"].includes(b);

```

```

const isOp = (b) => ["+", "-", "⌫", "÷"].includes(b);
const isEq = (b) => b === "=";
const isFunc = (b) => ["c", "μ", "sin", "deg", "Ac", "⌫"].includes(b);

return (
  <div style={{ minHeight: "100vh", display: "flex", alignItems: "center", justifyContent: "center", background: isDark ? "#0e0e1a" : "linear-gradient(145deg,#ddeeff,#cfcfcf)", padding: 20px }}>
    <div style={{ width: 300, borderRadius: 36, overflow: "hidden", background: t.cardBg, boxShadow: t.cardShadow, transition: "all 0.4s" }}>
      <div style={{ display: "flex", justify-content: "space-between", alignItems: "center", padding: "18px 20px 10px" }}>
        <span style={{ fontSize: 11, letterSpacing: 2.5, textTransform: "uppercase", fontWeight: 600, color: t.labelColor }}>Calculator</span>
        <button
          onClick={() => setIsDark(!isDark)}
          style={{ width: 46, height: 26, borderRadius: 13, border: "none", cursor: "pointer", background: isDark ? "#2a2a4a" : "#d4e0f8", position: "relative", transition: "background-color 0.3s" }}>
            <div style={{ position: "absolute", top: 4, left: isDark ? 24 : 4, width: 18, height: 18, borderRadius: "50%", background: "#4dabf7", transition: "left 0.3s" }}>
            </div>
          </button>
      </div>
      <div style={{ margin: "0 16px 8px", borderRadius: 22, background: t.displayBg, padding: "14px 20px 18px", minHeight: 110, display: "flex", flexDirection: "column", justify-content: "flex-end", align-items: "center" }}>
        <div style={{ fontSize: 13, color: t.exprColor, fontFamily: "monospace", minHeight: 18 }}>{expression || "0.000000"}</div>
        <div style={{ fontSize: display.length > 10 ? 22 : display.length > 7 ? 28 : 38, fontWeight: 700, color: t.displayText, fontFamily: "monospace", wordBreak: "break-all", textOverflow: "ellipsis" }}>{display}</div>
      </div>
      <div style={{ display: "grid", gridTemplateColumns: "repeat(4,1fr)", gap: 8, padding: "6px 16px 4px" }}>
        {["Ac", "⌫", "/", "*"].map((btn) => (
          <button
            key={btn}
            onClick={() => {
              if (btn === "Ac") handleClear();
              else if (btn === "⌫") handleBackspace();
            }}
          >{btn}</button>
        ))}
      </div>
    </div>
  </div>
)

```

```

139         else if (btn === "/" ) handleOperatorClick("/");
140         else if (btn === "") handleOperatorClick("");
141     })
142     style={{ height: 36, borderRadius: 10, fontSize: 13, fontWeight: 600, border: "none", cursor: "pointer", background: t.fun
143     >
144     {btn}
145     </button>
146     )}}
147 </div>
148
149 /* Main keypad */
150 <div style={{ display: "grid", gridTemplateColumns: "repeat(4,1fr)", gap: 10, padding: "8px 16px 20px" }}>
151 {mainButtons.map((btn, i) => {
152     const active = operator === btn && waitingForSecond;
153     let bg = t.numBg;
154     let color = t.numText;
155     let shadow = t.numShadow;
156     if (isOp(btn)) { bg = active ? "#4dabf7" : t.opBg; color = active ? "white" : t.opText; shadow = t.opShadow; }
157     if (isEq(btn)) { bg = "#4dabf7"; color = "white"; shadow = "0 4px 18px rgba(77,171,247,0.45)"; }
158     return (
159     <button
160     key={btn + i}
161     onClick={() => handleButtonClick(btn)}
162     style={{
163         height: 62, borderRadius: 18, fontSize: 21, fontWeight: 600, border: "none", cursor: "pointer",
164         display: "flex", alignItems: "center", justifyContent: "center",
165         fontFamily: "monospace", background: bg, color, boxShadow: shadow,
166         transition: "all 0.15s", transform: "scale(1)"
167     }}
168     onMouseDown={e => e.currentTarget.style.transform = "scale(0.91)"}
169     onMouseUp={e => e.currentTarget.style.transform = "scale(1)"}
170     onMouseLeave={e => e.currentTarget.style.transform = "scale(1)"}
171     >
172     {btn}
173     </button>

```

```

1     {mainButtons.map((btn, i) => {
5         })}
6     </div>
7 </div>
8 </div>
9 );
10 };
11
12 const theme = {
13     light: {
14         cardBg: "rgba(244,246,251,1)",
15         cardShadow: "0 20px 60px rgba(80,160,240,0.16), 0 4px 16px rgba(80,160,240,0.08)",
16         labelColor: "#a0b4cc",
17         displayBg: "ffffff",
18         displayText: "#1a2a3a",
19         exprColor: "#a0b8cc",
20         funcBg: "#e8f0fb",
21         funcText: "#4a7ab5",
22         numBg: "ffffff",
23         numText: "#1a2a3a",
24         numShadow: "0 2px 8px rgba(100,150,220,0.13)",
25         opBg: "#e8f2fd",
26         opText: "#2a7fd4",
27         opShadow: "0 2px 8px rgba(60,160,210,0.14)",
28     },
29     dark: {
30         cardBg: "#1c1c2e",
31         cardShadow: "0 20px 60px rgba(0,0,0,0.55), 0 4px 16px rgba(0,0,0,0.4)",
32         labelColor: "#555875",
33         displayBg: "#13132a",
34         displayText: "#e8eaf6",
35         exprColor: "#4a4a6a",
36         funcBg: "#252545",
37         funcText: "#6a6aaa",
38         numBg: "#252545",
39         numText: "#e8eaf6",
40         numShadow: "0 2px 8px rgba(0,0,0,0.3)",
41         opBg: "#1e2040",
42         opText: "#5aa8f0",
43         opShadow: "0 2px 8px rgba(0,0,0,0.25)",
44     },
45 };
46
47 export default Calculator;

```

